Who Wrote It?

Identifying LLMs from Their Responses

Team leader: George Dayoub
Team member: Atharv Madne

1 Introduction

The goal of this project was to see if a CNN or LSTM could identify which LLM wrote a response. I made my own dataset by sending 100 prompts to three LLMs through the NVIDIA API. This created 300 labeled responses. I tested both classifiers with the prompt only, the response only, and both together.

The two main questions were:

- RQ1: Can we identify which LLM generated a response?
- RQ2: Does the user's prompt help identify the LLM?

2 Dataset and training setup

The three models were GPT OSS (openai/gpt-oss-20b), Nemotron (nvidia/nemotron-3.5-lightning-30b-a3b), and GLM (z-ai/glm-5.3-flash). The prompts covered explanations, coding, math, creative writing, anime, Elden Ring, and other topics. Every model answered the same prompts.

I used 64 prompts for training 16 for validation and 20 for testing. Since each prompt had three responses thats 192 training responses and 48 validation responses, and 60 test responses. All three answers to one prompt stayed in the same split so the test set can have new prompts.

The text was changed into tokens and then numbers so PyTorch could use it. Both models learned their own word embeddings. The CNN searched for short word patterns. The LSTM read the sequence in both directions. Both used dropout, cross entropy loss, and batch size of 16, and eight epochs. I trained them on my laptop using CUDA.

3 Results

3.1 RQ1: Can we identify which LLM generated a response?

For RQ1, the classifiers received only the generated response. The CNN got 40 of 60 test responses correct, while the LSTM got 25 of 60 correct. With three balanced classes, always guessing one LLM would get 20 of 60 correct.

Classifier	Correct	Accuracy	Macro F1
CNN	40/60	66.7%	0.662
LSTM	25/60	41.7%	0.391

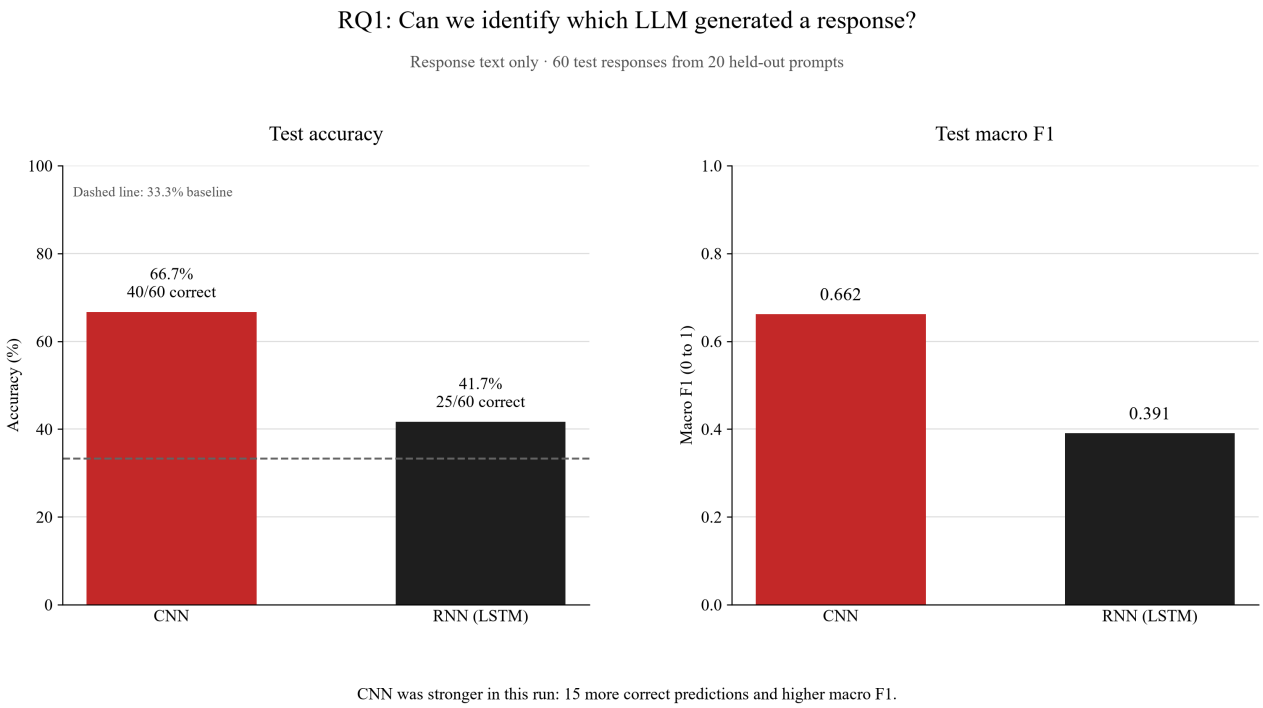


Figure 1: CNN and LSTM results using only the generated response.

The CNN performed better on both accuracy and macro F1. This means the response text contained patterns that helped identify the writer in this dataset. It does not mean the model would get the same score on other LLMs or a different group of prompts.

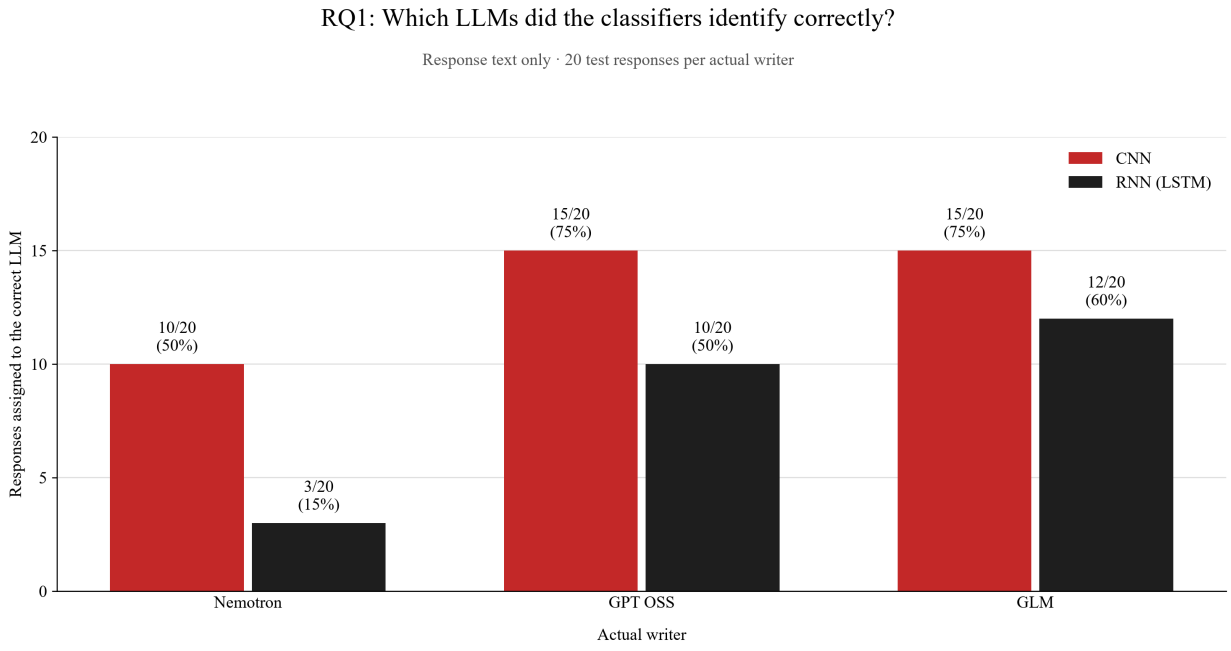


Figure 2: Correct predictions for each actual LLM using response text only.

The CNN correctly identified 10 of 20 Nemotron responses and 15 of 20 responses from both GPT OSS and GLM. The LSTM identified 3 Nemotron, 10 GPT OSS, and 12 GLM responses. Nemotron was the hardest writer for both classifiers.

3.2 RQ2: Does the user's prompt help identify the LLM?

Text used	CNN accuracy	CNN F1	LSTM accuracy	LSTM F1
Prompt only	33.3%	0.290	33.3%	0.317
Response only	66.7%	0.662	41.7%	0.391
Prompt + response	66.7%	0.652	45.0%	0.452

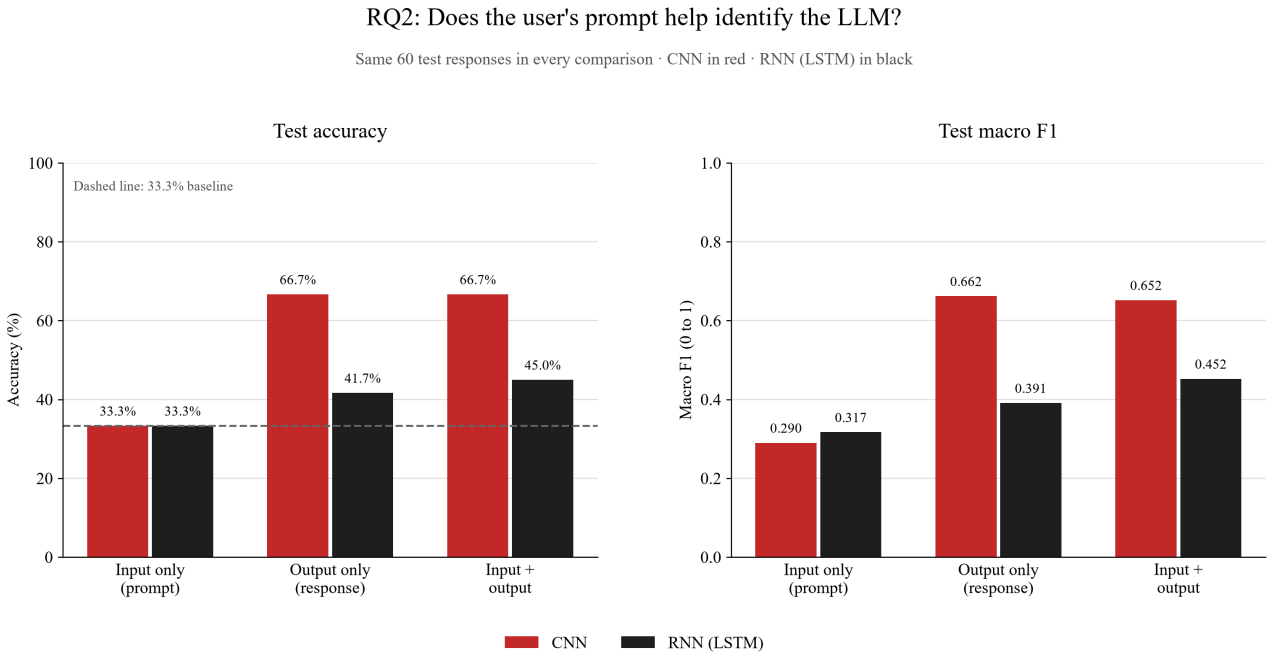


Figure 3: Comparison of prompt only, response only, and prompt plus response.

The prompt only classifiers both got 33.3%. This makes sense because the exact same prompt was paired with all three LLM labels. The prompt by itself could not show which model answered it. Adding the prompt didn't improve CNN accuracy. It improved the LSTM from 25 to 27 correct answers, which was only two more out of 60.

4 Deeper analysis

The CNN had the same 66.7% accuracy with response only and prompt plus response, but it didn't make the same guesses. Adding the prompt fixed nine mistakes and also caused nine new mistakes. For the LSTM, it fixed 13 mistakes and caused 11 new ones, giving it a net gain of two correct answers. This showed me that the total score can stay almost the same even when individual predictions change.

The LSTM training loss dropped from 1.114 to 0.137, but its response-only test accuracy was only 41.7%. It learned the training data much better than it handled new prompts. This helped me understand why training loss alone is not enough to judge a model.

Response length may also have helped the classifiers. Nemotron averaged about 81 words per response, while GPT OSS and GLM averaged about 103. The CNN may have noticed length or formatting along with word choice. I didn't test this directly, so I can't say that length caused the result.

5 Lessons and experience

At the start of the project, my teammate Atharv and I wanted to make some of our questions about things we actually liked. We play Elden Ring a lot so we added questions about that. The Penn State White Out had also just happened, which gave us the idea to include questions about Penn State football. We still used other topics like coding, explanations, math, and creative writing so the dataset would have some variety.

We really wanted to create our own dataset because it let us customize the project instead of using questions that did not mean anything to us.We saw multiple datasets on hugginface but wanted to try with our own. It was more interesting to look through responses about topics we knew and see how each LLM answered them. The NVIDIA API helped us send the same questions to all three LLMs and save their responses. This made the data feel more personal to us while still keeping the comparison fair.

I also learned that collecting the text was only part of the work. I had to organize the CSV, check the labels, keep the splits fair, and turn the words into numbers before the models could train. It ended up almost taking more than seeting up the models and training them. I also learned that the LSTM was much slower to train than the CNN, which made me appreciate the CNN more.

Getting everything working in PyTorch was a little difficult at first. Once CUDA was working on my laptop, I could train faster and watch the loss and validation scores change after each epoch. I had used CNNs in a few small projects before, so I was a little familiar with them. The LSTM was new to me, and it was intersting seeing how it handled and comparing it with the CNN. I learned alot from seeing the models make different mistakes on the exact same test data so we could see how they differ.

Next time, I would collect more prompts and plan the categories more evenly. I would include more examples of reasoning, coding, summaries, explanations, and creative writing with different difficulty levels. I would still send every prompt to all three LLMs so the comparison stays fair.

I would also test the response length idea. First I'd try a simple classifier that only sees the number of words. Then I would shorten the responses to similar lengths and train the CNN and LSTM again. This would help show whether the classifiers learned writing differences or mostly noticed that one model gave shorter answers.

6 Limitations and conclusion

This project used only 100 prompts, one split, and one training run. The test set had 60 responses from 20 prompts, so a few predictions could noticeably change the scores. More prompts and repeated runs would give stronger results.

For this dataset, the answer to RQ1 is yes: the CNN identified the writer better than the 33.3% baseline and reached 66.7% accuracy. For RQ2, the user's prompt didnt improve the CNN and only helped the LSTM by two answers. The response itself was much more useful than the prompt for identifying the LLM.